Desarrollo de un conjunto de algoritmos para el estudio de teoremas y expresiones del campo de la aritmética modular

Pablo Tuñón Laguna Jorge Vančo Sampedro

Índice

Introducción 3
Algoritmos 4
Verificación de primalidad 4
Lista de primos entre 2 números 6
Factorización de un número 7
Máximo común divisor 8
Verificación de números coprimos 9
Potencia módulo p 10
Inversa módulo p 11
Símbolo de Legendre 13
Resolver sistemas de congruencias 14
Algoritmos extra 17
Bibliografía 18

Introducción

La aritmética modular es una disciplina de las matemáticas que se encarga del estudio del módulo, es decir el resto de una división y de sus propiedades en relación a otros números, es por ello que es un campo de estudio que tiene sus raíces en la teoría de números.

A lo largo del siguiente escrito se van a analizar la implementación en Python de algunas de las propiedades, teoremas y funciones más usados de la aritmética modular, con el fin de desarrollar un profundo conocimiento sobre las mismas. Al realizar dicha librería de comandos se ha obtenido un conocimiento considerablemente mayor al que se genera al aprender dichos conocimientos en un entorno teórico ya que el rigor conceptual para el correcto funcionamiento debe ser mayor.

En el siguiente estudio no sólo se haya recogido la explicación de las distintas implementaciones, se ha intentado llegar más lejos, tratando de mejorar el tiempo que tardan en completarse las distintas funciones de la librería, a través de la mejora de los algoritmos ya programados y de la completa sustitución de otros ya que se ha encontrado y comprendido uno más eficiente.

Finalmente, para una mayor difusión y una simplificación del uso de la librería de algoritmos, se ha desarrollado una interfaz gráfica que permite la aplicación de las funciones desarrolladas a los conjuntos de números que se quiera, el resultado se muestra en una parte de la misma. Esta interfaz también permite una ejecución por lotes, basta con escribir el nombre del fichero de texto en el espacio indicado y el programa abrirá el txt en caso de que exista y realizará las funciones pedidas almacenando el resultado en un documento de texto nuevo.

Algoritmos

Verificación de primalidad

Aquel número que es divisible por si mismo y por el 1 es un número primo, por ello se sabe que un primo no puede ser par, ya que implicaría que es múltiplo de 2 y por tanto no primo. Con este planteamiento se realizó una primera implementación de la función que verifica la primalidad de un número, comprobando el resto de dividir este por los números impares contenidos entre 3 y la raíz del número por consecuencia del teorema fundamental de la aritmética. Sin embargo, este algoritmo (que realmente no era más que una versión simplificada de la criba de Eratóstenes) no era para nada eficiente a nivel de consumo de recursos y tiempo, por lo que se buscaron otros.

En el proceso de búsqueda se encontró el algoritmo de Miller Rabin, en él se expresa el número que se quiere comprobar como primo de la siguiente manera:

$$n - 1 = (2^s) \times d$$

En la expresión anterior n representa el número que se quiere verificar primo (por la definición de primo debe ser impar), s es un número natural y d es un número impar. A continuación, sobre s y d se aplican unas propiedades un número de veces que probabilísticamente extraen si el número es primo o no:

$$a^d \equiv 1 mod(n)$$

En la propiedad anterior a es un número entre 2 y n – 1, esta se basa en el pequeño teorema de Fermat, que establece que si se posee un cierto número primo a y se encuentra otro coprimo p (aquellos cuyo máximo común divisor es 1) al elevar el al elevar a a la potencia p – 1 se obtendrá un resto igual a 1, que es lo que significa que a elevado a p-1 sea congruente con 1 módulo p. Esta propiedad se aplica a Miller Rabin de tal manera que si se elige un número a coprimo con n y se calcula a elevado a n-1 módulo n en caso de que se obtenga un resto igual a 1 hay probabilidades de que n sea primo.

A continuación, se enuncia la segunda propiedad:

$$a^{2^r \times d} \equiv -1 mod(n)$$

En la propiedad anterior r es un número mayor o igual que 0 pero menor que s(recordar la forma en la que se ha expresado n), esta se basa en el hecho de que los únicos residuos cuadrados modulo n son 1 y -1, ya que si se eleva un número al cuadrado repetidamente en busca de un valor r que cumpla con la propiedad, en caso de que la cumpla para distintas a's, habría bastantes probabilidades de que n fuera primo.

Miller Rabin es un algoritmo probabilista y se considera que sería más adecuado encontrar uno con 0% tasa de fallo e incertidumbre, por ello se continuó buscando y se encontró una versión determinista del mismo que se basa en la hipótesis de rieman generalizada, la cual por su complejidad y debido a que ni ha sido refutada ni ha sido demostrada no se encuentra recogida en dicho escrito. La modificación que realiza reside en el número que puede ser la a, en lugar de ser un número entre 2 y n-1 es un número entre 2 y la siguiente expresión:

$$\underline{\min\left(n-2, \lfloor 2 \times \ln(n)^2 \rfloor\right)}$$

Aparte ciertos matemáticos han hecho comprobaciones respecto a si n es relativamente pequeño (25 cifras) las a's que verifican su primalidad de forma determinista que n es primo y las han tabulado, por lo que dichas listas también se usan reduciendo aún más el tiempo sin dejar de ser un algoritmo robusto.

Lista de primos entre 2 números

La función desarrollada es bastante sencilla a nivel conceptual, busca obtener los primos que existen entre 2 números. En un principio se consideró el desarrollo de la criba de Eratóstenes en la cual se van eliminando los múltiplos de cada uno de los primos en orden hasta que únicamente quedan en la lista aquellos que son primos y posteriormente se cogen aquellos que están en el intervalo. Sin embargo, para intervalos de números relativamente grandes (números de 10 cifras) el tiempo de procesado de la función era verdaderamente grande. Esto se debe a que primero hay que comprobar si un número es primo y posteriormente eliminar todos aquellos que sean múltiplos del mismo.

En un segundo planteamiento de la función, se usó la versión determinista de Miller Rabin para recorrer el intervalo de números entre los dos escogidos añadiendo a una lista aquellos que son primos. Sin embargo, para números con una gran diferencia de cifras, como por ejemplo 2 frente a 8 cifras, el tiempo de ejecución también era considerablemente largo.

Finalmente se valoró una mejora de la criba de Eratóstenes, la criba de Euler. Dicha modificación después de haber generado una lista con todos los números entre los dos deseados, inicia igual que la de Erastótenes, eliminando los múltiplos de 2 y los múltiplos de 3, pero trabaja directamente en el intervalo de los dos números marcados. A continuación, genera los números primos menores que la raíz del mayor del intervalo y divide y elimina sus múltiplos en el intervalo, de esta manera se consumen menos recursos temporales y físicos.

Factorización de un número

Para la factorización de un número también se probaron múltiples métodos. En un primer momento se probó con una visión básica, se usó una función recursiva para comprobar si el número con el que se trabaja es primo y en caso que no lo sea se divide por un número primo cada vez mayor hasta que su resto sea 0, en ese momento, se envía el cociente como nuevo número para factorizar a la función de forma recursiva. También se envía el primo que ha dividido al número original, de tal manera que se conozcan los factores del número original y el primo a partir del cual hay que comprobar en la siguiente iteración de la función recursiva. Sin embargo, este método de factorización, aunque sencillo es algo ineficiente.

GNFS ó "General Number Field Sieve" es la forma de factorización usada para números de más de 100 cifras, ya que muestra unos tiempos bastante buenos en relación con el tamaño del número y otros algoritmos probados. Este método también fue considerado, pero al intentar comprenderlo, se observó que la tasa de acierto era de 2/3, lo cual para números tan grandes es muy valioso, sin embargo, recordemos que en la implementación de los teoremas se busca no sólo eficiencia temporal, también se aboga por el rigor de la solución. Tras considerar GNFS se reparó en QS, aunque en la teoría es el mejor algoritmo de factorización para números con menos de 100 cifras, al realzar una serie de pruebas con los benchmarks proporcionados y una implementación de GitHub, se observó que no eran unos tiempos adecuados, aunque es posible que la implementación del algoritmo no fuese la más optima.

QS se abandonó ya que paralelamente en el proceso de investigación se encontró un nuevo algoritmo que parecía más sencillo de implementar y no tan costoso a nivel temporal, Rho de Pollard, concretamente su versión determinista. El algoritmo inicia con dos valores iniciales establecidos en 2, posteriormente se la siguiente función de mapeo sobre x e y:

$$g(x) = (x^2 + 1)\%n$$

En la anterior función n es el número a factorizar, y se usa dicha función de mapeo debido a que explora distintas trayectorias en espacio de búsqueda de factorización de un número compuesto, se sabe que hay posibilidades de haber encontrado un factor si los resultados de g(x) y g(g(y)) (ya que se usa el algoritmo de la liebre y la perdiz, explorado la y más rápido que la x) son coprimos.

En caso de que sean coprimos, hay una posibilidad de que el máximo común divisor sea un factor del número. Para ello se comprueba si el número obtenido es el cuadrado de un primo, en caso de que lo sea se devuelve el primo, en caso de que no lo sea se verifica si el máximo común divisor en sí es primo y en caso de serlo también se devuelve como factor, si no lo fuera se devolvería el vacío. El proceso restante es sencillo, mediante un bucle se realiza rho de pollard sobre el número a factorizar guardando los factores y dividiendo el número por su factor una vez encontrado, repitiendo el proceso hasta que el número a factorizar sea un número primo.

Máximo común divisor

El programa realizado para implementar este cálculo es correspondiente con el algoritmo de Euclides. En él se poseen dos números a y b tal que a es mayor o igual que b, el proceso que lleva a cabo es el siguiente. Se verifica que ninguno de los dos números es 0, ya que en ese caso hacer el máximo común divisor de ambos números sería equivalente al número no nulo, posteriormente se divide a entre b calculando el resto. En caso de que este sea nulo, se devuelve b, finalmente si el resto no es nulo se aplicará:

$$a = \max(resto, b)\ y\ b = \min\ (resto, b)$$

El proceso anterior se repetirá hasta que b sea igual a 0, es decir que al hacer el mínimo de la anterior división entre el divisor y el resto se obtiene 0, en ese momento se devuelve el actual a o el anterior b (el anterior b es el máximo entre 0 y él mismo).

Para optimizar este algoritmo, se implementó la mejora señalada en el artículo sobre mejorar el algoritmo de Euclides (S.K. Lucas, 2007). Dado que mcd(a,b) = mcd(a, b – a), se puede usar la resta en cada paso del algoritmo de Euclides para acelerar este, dándose dos pasos en uno.

También se implementó una función que obtiene el máximo común divisor de más de dos números, esta se basa también en el algoritmo de Euler y en el principio de divide y vencerás. Teniendo los números en una lista, realiza el máximo común divisor entre los dos primeros, a continuación, sustituye los dos primeros por el su máximo común divisor. Se repite el proceso para los posteriores integrantes de la lista, hasta que únicamente queda un número en la lista, este será el máximo común divisor de todos los números que al inicio de la función se encontraban en la lista.

Verificación de números coprimos

Dos números son coprimos en caso de que su máximo común divisor sea 1. Por ello a la función se le proporcionan dos números, n y m y se disponen tal que n es el mayor y m es el menor. En caso de que sean idénticos, los números no pueden ser coprimos ya que su máximo común divisor sería distinto de 1. A continuación en pos de una reducción del tiempo, se dividen n y m (que ahora vale n-m) por los primeros números primos, de tal manera que, si en algún momento ambos números son divisibles por el mismo primo, se deje de comprobar si son primos. Finalmente se aplica el algoritmo de Euclides para obtener el máximo común divisor de n y m, de tal manera que si este es igual a se devuelve un booleano verdadero y si es distinto se devuelve un booleano falso.

Potencia módulo p

En un primer momento, se implementó como una base elevada a un exponente de la cual posteriormente se hacía el módulo. Sin embargo, al realizar unas pruebas fue evidente la ineficiencia de la función, ya que para bases y exponentes grandes podía generar números inmanejables por Python (mayores a 19 cifras) aparte que para ello se hacía uso de bastantes recursos temporales y de memoria computacional. Por todo ello se buscó una mejor forma de realizar dicho cálculo.

El algoritmo desarrollado comienza rechazando casos trivilaes, como el hecho de que el módulo sea 1 en cuyo caso el resultado va a ser siempre 0, o que el módulo sea 0, en cuyo caso no se podría llevar a un acabo y se lanzaría un error al programa principal, que imprime por pantalla NE. A continuación, se verifica que el exponente no sea negativo, en caso de que lo sea se aplicaría un valor absoluto sobre el número y se haría que la base fuera el inverso de la base módulo p. Posteriormente se itera una serie de operaciones hasta que el exponente sea igual o menor que 0, si el exponente módulo 2 es igual a 1, el resultado de toda la operación se multiplica por la base y se realiza el módulo p de la misma, siempre se divide el exponente por dos y se eleva la base al cuadrado haciendo posteriormente el módulo de la misma. Todo ello permite que se vaya multiplicando el resultado por la base gradualmente y aplicando el módulo, de tal manera que no se generan números tan grandes ni se hace uso de tanta memoria.

Inversa módulo p

Esta implementación sirve para calcular el inverso multiplicativo de un número módulo otro, es decir para resolver el siguiente tipo de ecuaciones de la siguiente manera:

$$n \times x \equiv mod(p) \leftrightarrow n \times n^{-1} \times x \equiv n^{-1} mod(p) \leftrightarrow x \equiv n^{-1} mod(p)$$

En las expresiones anteriores, n y p son ambos números enteros, y la expresión de la derecha representa a x como el inverso multiplicativo de n módulo p, que es lo que se encarga de calcular la función, esta es una implementación del algoritmo de Euclides extendido.

En primer lugar, se asegura que n sea positivo y menor que p realizando la operación módulo sobre el mismo. Posteriormente se itera hasta que el resto sea 0, de tal manera que se obtiene el máximo común divisor de ambos números como combinación de n multiplicado por una constante y p multiplicado por otra constante. Finalmente se comprueba si el máximo divisor es igual a uno (es decir, que son coprimos) y en caso de que lo sea se devuelve el coeficiente de n módulo p, obteniendo así el inverso multiplicativo del número n con el módulo p.

Phi de Euler

La función phi Euler tanto a nivel conceptual como a nivel de implementación no tiene una gran complicación, es una función que cuenta el número de coprimos enteros positivos que existen menores a n.

Para ello, en primer lugar se eliminan los casos imposibles, que son todos aquellos números negativos, ya que la función únicamente funciona con valores positivos, en caso de tener un valor negativo el programa principal imprimiría por pantalla NE. A continuación, extrae los factores del número n y se multiplica el n por 1 – el inverso de cada uno de los factores y esto devuelve el número de coprimos inferiores a n, esto se debe por la siguiente propiedad:

$$\varphi(p^k) = (p-1) \times p^{k-1}$$

Símbolo de Legendre

El símbolo de Legendre devuelve 0, 1 ó -1 en función de la relación de n y p, siendo estos un entero y un número primo respectivamente. En caso de que n sea congruente con 0 mod p, se devuelve un 0, en caso de que n sea congruente con n cuadrado módulo p se devuelve un 1 y en caso de que no lo sea, se devuelve -1.

En caso de que el módulo sea 0, el programa principal imprimirá NE, ya que no existe la operación a realizar. A continuación, se eliminan dos casos en los que se puede simplificar incluso más, para n igual a -1 o a 2:

$$\left(\frac{-1}{p}\right) = (-1)^{\frac{p-1}{2}} \; y \; \left(\frac{-1}{p}\right) = (-1)^{\frac{p^2-1}{8}}$$

Para el resto de casos, directamente se realiza la potencia de n elevado a (p – 1)//2 módulo p, si el resultado es mayor que 1, se devuelve -1, en caso contrario se devuelve el resultado de la función

Resolver sistemas de congruencias

Esta función resuelve sistemas de la forma:

$$\begin{cases} a_1 x \equiv \mathrm{r}_1 mod(p_1) \\ \quad\quad\quad ... \\ a_2 x \equiv \mathrm{r}_\mathrm{n} mod(p_n) \end{cases}$$

Se ha hecho uso del Teorema de congruencia lineal repetidas veces para resolver los sistemas, como se indica en el artículo de sistemas de congruencias (Wikipedia, s.f.). Se ha elegido esta implementación dado que funciona aun cuando todos los $p_i \mid i = 1, \dots, n$ no son coprimos dos a dos.

Se resuelve una ecuación, queda $x = p_1 k + a_1^{-1} r_1$, sustituyendo esto en las sucesivas ecuaciones y despejando, podemos llegar a la solución.

En la implementación en Python (Figura *1*. Resolver_sistema_congruencias), se han usado números complejos para representar el coeficiente de k (la parte imaginaria) y el resto (la parte real).

Mientras se recorren todas las ecuaciones, se comprueba que $mcd(a_1, p_1) | r_1$. Puesto que, si no es el caso, es incompatible la ecuación, y se lanza una excepción.

```python
def resolver_sistema_congruencias(
    alist: List[int], blist: List[int], plist: List[int]
) -> Tuple[int, int]:
    """
    Dadas tres listas de números enteros [a_1,...,a_n], [b_1,...,b_n] y [p_1,...,p_n], resuelve el sistema de congruencias

    a_i * x = b_i (mod p_i)   i=1,...,n

    devolviendo un entero r y un módulo m tales que las soluciones del sistema corresponden a todos los enteros
    x congruentes con r módulo m.

    Args:
        alist (List[int]): Lista de coeficientes de la variable x, [a_1,...,a_n].
        blist (List[int]): Lista de términos independientes [b_1,...,b_n].
        plist (List[int]): Lista de módulos [p_1,...,p_n]

    Returns: (r,m)
        r (int): Entero entre 0 y m-1.
        m (int): Entero positivo, módulo de la solución.

    Raises:
        IncompatibleEquationError: Si no es posible resolver el sistema.
    """

    X = None
    for a, b, p in zip(alist, blist, plist):
        if b % mcd(a, p) != 0:
            raise IncompatibleEquationError

        if X is not None:
            k = a * X
            a = k.imag % p
            b = (b - k.real) % p

        mcd_ecuacion = mcd_n([a, b, p])

        if mcd_ecuacion > 1:
            a, b, p = a // mcd_ecuacion, b // mcd_ecuacion, p // mcd_ecuacion

        if b % mcd(a, p) != 0:
            raise IncompatibleEquationError
        x = p * 1j + (inversa_mod_p(a, p) * b) % p

        if X is not None:
            X = X.imag * x + X.real
        else:
            X = x

    return int(X.real), int(X.imag)
```

Figura 1. Resolver_sistema_congruencias

Raíz módulo p

Tras investigar el algoritmo de Cipolla (Wikipedia, s.f.) y no llegar a resultados rápidos, se optó por implementar el algoritmo propuesto en el siguiente artículo: (Rajeev Kumar, 2021).

El algoritmo se basa en que solo hay dos posibles raíces para cualquier elemento $\alpha \in Z_p^*$, y que la raíz de $1 \in \{1, p-1\}$. Esto permite construir un conjunto con $2^{(r-1)}$ elementos como mucho (donde $p - 1 = 2^r * d$; $r, d \geq 1$ y d un entero impar), de los cuales uno debe ser la raíz.

```python
def raiz_mod_p(n: int, p: int) -> list:
    """Encuentra, si existe, una raíz cuadrada para un entero n módulo un número primo p.

    Args:
        n (int): Entero del que se desea hallar la raíz.
        p (int): Módulo. Se asume que es un número primo.

    Returns:
        int: Entero x entre 0 y p-1 tal que x^2 = n (mod p).

    Raises:
        IncompatibleEquationError: Si no es posible hallar dicha raíz.
    """

    if legendre(n, p) == -1:
        return []
    n %= p
    if n == 0 or n == 1:
        return (n, -n % p)

    if p == 2:
        return (n, -n % p)

    a = 2
    while legendre(a, p) == 1:
        a += 1

    n_pow = p - 1
    a_pow = 0

    while n_pow % 2 == 0:
        n_pow //= 2
        a_pow //= 2

        if potencia_mod_p(n, n_pow, p) % p * (potencia_mod_p(a, a_pow, p) % p) == p - 1:
            a_pow += (p - 1) // 2
    n_pow = (n_pow + 1) // 2
    a_pow //= 2

    sqrt = potencia_mod_p(n, n_pow, p) * potencia_mod_p(a, a_pow, p) % p
    sqrts = [sqrt, -sqrt % p]
    sqrts.sort()
    return tuple(sqrts)
```

Figura 2. Raiz_mod_p

Ecuación cuadrática

Para calcular las soluciones de una ecuación cuadrática (R.C.Dailea, s.f.) del estilo: $ax^2 + bx + c \equiv 0 \bmod(\mathrm{p})$. Completando cuadrados y despejando se llega a que $x \equiv (2a)^{-1} * \left(-b + \sqrt{b^2 - 4 * a * c}\right)\ mod(p)$.

Existen soluciones si existe la raíz de $b^2 - 4 * a * c$ mod(p), cuyas 2 raíces se calculan con el algoritmo descrito anteriormente.

```python
def ecuacion_cuadratica(a: int, b: int, c: int, p: int) -> Tuple[int, int]:
    """
    Halla, si es posible, las dos posibles soluciones de la ecuación cuadrática ax^2+bx+c=0 (mod p).
    Devuelve una tupla con las dos raíces (distintas o una misma raíz repetida en caso de ser doble).

        Args:
            a (int): Coeficiente de x^2.
            b (int): Coeficiente de x.
            c (int): Término independiente.
            p (int): Módulo. Se asume que es un número primo.

        Returns: (x1,x2)
            x1 (int): Primera solución. Entero entre 0 y p-1.
            x2 (int): Segunda solución. Entero entre 0 y p-1.

        Raises:
            IncompatibleEquationError: Si no es posible resolver la ecuación.
    """

    if mcd(2 * a, p) != 1:
        raise IncompatibleEquationError
    solutions = []
    square = (b**2 - 4 * a * c) % p
    try:
        for sol in raiz_mod_p(square, p):
            solutions.append((inversa_mod_p(2 * a, p) * (-b + sol)) % p)
        solutions.sort()
        return solutions
    except IncompatibleEquationError as e:
        raise IncompatibleEquationError(e)
```

Figura 3. Ecuacion_cuadratica

Algoritmos extra

Algoritmo de cipolla

Se ha mantenido el algoritmo de Cipolla (Wikipedia, s.f.) (Figura *5. C*ipolla) para calcular la raíz módulo p. Se ha usado una función auxiliar (Figura *4.* cipolla_mult) para calcular las multiplicaciones necesarias en este algoritmo.

Como primer paso, se busca un entero a tal que $a^2 - n$ no sea un cuadrado. Las raíces se calculan como $x_1 \equiv \left(a + \sqrt{a^2 - n}\right)^{\frac{p+1}{2}} mod(p)$, siendo la otra solución $x_2 \equiv -x \, mód(p)$.

```python
def cipolla_mult(a: int, n: int, p: int) -> int:
    """
    Función para calcular las multiplicaciones usadas en el algoritmo de Cipolla
    se multiplica sucesivas veces (a*1 + w^0.5) y se busca mantener los coeficientes de w**0.5 y de 1

    Args:
        a(int): número que no es un no residuo cuadrático módulo p
        n(int): el número del que se quiere calcular la raíz módulo p
        p(int): el número primo

    Returns:
        new_a(int): una raíz, el coeficiente de 1 en el resultado final de las multiplicaciones (new_a + coef_w * w**0.5)
    """
    f_exp = (p + 1) // 2
    exp = 1
    w = a**2 - n

    new_a = a
    new_coef_w = 1
    while 2 * exp < f_exp:
        new_a, new_coef_w = (new_a**2 + new_coef_w**2 * w) % p, (
            2 * new_a * new_coef_w
        ) % p
        exp *= 2

    while exp < f_exp:
        new_a, new_coef_w = (a * new_a + new_coef_w * w) % p, (
            a * new_coef_w + new_a
        ) % p
        exp += 1

    return new_a
```

Figura 4. cipolla_mult

```python
def cipolla(n, p):
    """
    Encuentra, si existe, una raíz cuadrada para un entero n módulo un número primo p.

    Args:
        n (int): Entero del que se desea hallar la raíz.
        p (int): Módulo. Se asume que es un número primo.

    Returns:
        int: Entero x entre 0 y p-1 tal que x^2 = n (mod p).

    Raises:
        IncompatibleEquationError: Si no es posible hallar dicha raíz.
    """
    n %= p
    if n == 0 or n == 1:
        return (n, -n % p)
    if not cuadratic_residue(n, p):  # Si n no es un cuadrado módulo p, no tiene raíces
        raise IncompatibleEquationError(
            f"\nLo sentimos, {n} no es un cuadrado módulo {p}."
        )
    phi = p - 1

    if potencia_mod_p(n, phi // 2, p) != 1:
        return
    if p % 4 == 3:
        x = potencia_mod_p(n, (p + 1) // 4, p)
        return (x, -x % p)
    a = 1
    while legendre(a**2 - n, p) != -1:  # Se busca una raíz cuadrada de n módulo p
        a += 1

    b = cipolla_mult(a, n, p)

    sols = [b, -b % p]
    sols.sort()
    return sols
```

Figura 5. Cipolla

Bibliografía

A continuación, se muestran los recursos usados:

- Wikipedia, n.d. *Miller-Rabin Primality test,* s.l.: s.n.
- Case M. [2003] *https://www.cs.umd.edu/~gasarch/TOPICS/factoring/NFSmadeeasy.pdf*
- Crampton P. [2019] *https://github.com/cramppet/quadratic-sieve*
- Alcalde A. [2017] *https://elbauldelprogramador.com/criptografia-101-fundamentos-matematicos-ii/*
- R.C.Dailea, n.d. *Quadratic Congruences, the Quadratic Formula, and Euler's Criterion,* s.l.: s.n.
- Rajeev Kumar, S. f. N. T. R., 2021. *A simple algorithm for finding square root modulo p,* Bengal, India: s.n.
- Wikipedia, n.d. *Símbolo de Legendre,* s.l.: s.n.
- S.K. Lucas, D. o. M. a. S., 2007. *Improving the Euclidean Algorithm,* s.l.: s.n.
- Wikipedia, n.d. *Cipolla's Algorithm,* s.l.: s.n.
- Wikipedia, n.d. *https://es.wikipedia.org/wiki/Teorema_de_congruencia_lineal,* s.l.: s.n.
- Lerch .D [2007] https://dlerch.blogspot.com/2007/10/factorizacin-ii-rho-de-pollard.html
- Monum A. [2023] https://www.educative.io/answers/what-is-extended-euclidean-algorithm